Before Fire Season

A Leader's Guide to Preventive Technical Health

The best time to address tech debt is when you're not in crisis.

You took the assessment and landed in a good place. Your systems are mostly healthy. Your team is shipping. Customers are not escalating at alarming rates. If you're reading this, things are probably working.

So why are we sending you a white paper?

Because "mostly healthy" is not the same as "safe." Healthy forests still accumulate dry brush. Healthy engineering organizations still accumulate tech debt. The difference between the ones that stay healthy and the ones that end up in crisis is whether they clear the brush before fire season, or wait until the wind picks up.

This paper is about building the discipline to do that clearing while you still have the luxury of choosing when and how.

Why Healthy Organizations Still Accumulate Debt

Tech debt is not a sign that something went wrong. It's a byproduct of building things under real constraints. You shipped a feature on deadline and cut a corner you planned to come back to. You chose the faster implementation because the business needed it now. You inherited a system from a team that no longer exists and never had time to fully understand it.

All of those are reasonable decisions made by reasonable people. The problem is not any single decision. The problem is that these decisions accumulate, quietly, and each one makes the next one slightly easier to justify. "We already have one workaround in that service, what's one more?" "The tests are already flaky, adding another test won't help." "Nobody's touched that code in a year, it's probably fine."

This accumulation is physics, not failure. Every organization that builds software experiences it. The question is whether you have practices in place to address it continuously or whether you're waiting for a crisis to force the conversation.

The Five Most Common Brush Patterns

After working with engineering organizations across industries, the same patterns show up over and over. These are the five most common forms of brush that accumulate in otherwise healthy codebases.

1. Test Gaps

The code works, but the tests don't prove that it works. Maybe coverage is low in the areas that matter most. Maybe the tests that exist are brittle and break for reasons unrelated to actual failures. Maybe the team has learned to ignore certain test results because they're unreliable.

Test gaps don't cause problems on their own. They cause problems when something changes. A refactor that would take a day with good test coverage takes a week when nobody trusts the test suite to catch regressions. Engineers move slower, not because they're less capable, but because they have no safety net.

2. Deployment Friction

You can release, but it takes more coordination than it should. There's a checklist. There's a person who needs to be available. There's a window. There are manual steps that everyone knows about but nobody has automated because they only take a few minutes each.

Those few minutes add up. More importantly, deployment friction changes behavior. Teams batch more changes into each release because releases are expensive. Larger releases are harder to debug when something goes wrong. The cycle reinforces itself. The friction does not have to be dramatic to be costly.

3. Undocumented Organizational Knowledge

The system works because specific people know how it works. Not because it's documented. Not because the architecture is self-explanatory. Because Sarah has been here for five years and knows why that configuration is set the way it is, and because Marcus is the only one who's ever deployed the billing service.

This is the brush pattern that feels the least urgent and carries the most risk. Everything is fine as long as Sarah and Marcus are there. The moment one of them leaves, takes an extended absence, or simply has a busy week when something goes wrong, the organization discovers how much of its operational knowledge lived in someone's head.

4. Deferred Upgrades

The framework is two major versions behind. The database is running a version that's approaching end of life. The infrastructure provider has been sending gentle emails about a migration deadline that's six months away.

Each of these is easy to defer. Upgrades are unglamorous work. They rarely produce visible features. The business case is hard to make when everything is currently working. But the cost of deferring grows over time, and at some point "we should upgrade" becomes "we can't upgrade without a major effort" becomes "we're stuck."

5. One-Customer Features

This is the brush pattern that's hardest to say no to, because it usually arrives attached to revenue.

A customer offers a significant deal that requires custom work. The organization says yes. The team builds it. The customer is happy. Revenue is captured. Everyone celebrates.

Then the customer leaves, and the custom code stays.

We've seen this play out in dozens of ways. A company spent three months of development team time building a custom page on their website for a brand partner who left a year later. Another organization built a custom implementation for a customer on the promise of a contract extension, invested six months of development work, and the customer asked to shelve the project and signed for only one more year, effectively costing the company $300,000 in development time they could not recover. Another company built a custom implementation for $1.5 million. Ten years later, that code is still sitting in the codebase. Nobody has touched it. Nobody fully understands it. It will sit there quietly until it breaks, at which point it becomes a crisis that pulls teams off feature work to go deal with something that should have been decommissioned years ago.

The pattern is always the same. The initial deal looks like revenue. What it actually creates is a maintenance obligation that outlasts the relationship. The custom code multiplies the number of paths a customer can go down, multiplies the code a developer has to sort through, and creates invisible ongoing costs that were never accounted for in the original deal. Five years after that million-dollar customer is gone, you're still carrying the weight of the thing you built for them.

The only healthy response to these opportunities is the one that starts with understanding what's actually being asked before saying yes. Not "how do we make it work?" but "what are

we committing to maintain, for how long, and at what cost?"

A few practices that protect you:

Account for the hidden costs upfront. When you evaluate a custom deal, don't just calculate the development cost against the contract value. Ask what happens if the customer is still on this custom implementation five years from now. Ask what happens if they leave after year one. Amortize or depreciate the development investment over time the same way you would any other capital expenditure. Run the math for both scenarios, they renew and they don't, and make the decision based on the full picture.

Build in a sunset period and stick to it. If you agree to custom work, define from the beginning when and how it will be decommissioned or absorbed into the standard product. Write it into the agreement. If you don't set that boundary at the start, you will not set it later, and you will be carrying that code indefinitely.

Treat custom implementations as depreciating assets. The $1.5 million custom build is not a $1.5 million asset. It's a $1.5 million asset that loses value every year while accumulating maintenance cost. If nobody is actively using it, maintaining it, or paying for it, it needs to be retired. The longer it sits untouched, the more expensive the eventual crisis when it breaks.

Assessing Severity Without Over-Investing

Not all brush needs to be cleared immediately. The goal is not to eliminate all tech debt. That's neither realistic nor a good use of resources. The goal is to understand what you have, know which pieces carry real risk, and address the ones that could turn into something worse.

A lightweight assessment looks like this:

1. **Inventory the known debt.** Ask your engineering leads to list the things they know are problems but haven't had time to address. Don't ask for solutions yet. Just the list.
2. **Categorize by blast radius.** For each item, ask: if this fails or becomes urgent, how many people are affected? Just the team? Customers? Revenue? The answer determines priority, not the technical severity.
3. **Identify the dependencies.** Some debt blocks other work. A framework upgrade might be a prerequisite for a security patch. A deployment automation might need to happen before you can safely increase release frequency. These dependencies change the order of operations.

4. **Estimate the cost of waiting.** For each item, ask: does this get harder to fix over time? If the answer is yes, that's a reason to act sooner. If the answer is no, it can wait.

This exercise should take a few hours across your engineering leadership, not weeks. If you find yourself building a massive spreadsheet, you've over-invested. The point is clarity, not completeness.

Building a Controlled Burn Practice

In forest management, a controlled burn is a deliberate, planned fire set under specific conditions to clear accumulated brush. It's not a response to a crisis. It's a practice that prevents one.

The engineering equivalent is scheduled, intentional debt reduction built into the normal rhythm of work.

What this looks like in practice:

Keep a maintenance lane. Reserve a consistent percentage of sprint capacity for debt reduction and maintenance work. Twenty percent is a good starting point. Some sprints you'll use all of it. Some sprints you won't need to. That's the point. When the system is healthy, that unused capacity flows back into feature work. When something needs attention, the lane is already there and nobody has to fight for permission to use it.

Here's what this actually produces. On a team that maintained this discipline, PagerDuty calls dropped to roughly one every six months. The system hummed. And because the maintenance lane kept things healthy, the team could dedicate 95% of their sprint time to new features. Most leaders assume that maintenance work takes away from feature development. In practice, *not* doing maintenance is what takes away from feature development, because you end up spending that time on emergency fixes, incident response, and working around problems that should have been addressed months ago.

Run periodic burn weeks. In addition to the ongoing maintenance lane, schedule periodic "burn weeks" where the team focuses entirely on clearing accumulated brush. This is different from the maintenance lane. The lane handles the steady trickle. Burn weeks tackle the backlog, the larger items that need focused, uninterrupted attention. Once a quarter works well for most teams. The cadence matters less than actually protecting the time when it comes.

Make the work visible. Debt reduction work should go through the same planning and review process as feature work. If it's invisible, it's the first thing that gets cut when priorities compete. Track it. Demo it. Show leadership what was addressed and why it mattered.

Let the team choose what to burn. The engineers closest to the code know which debt causes the most friction. Give them the capacity and let them prioritize. This builds ownership and ensures the work targets the things that actually slow people down, not the things that look worst on a slide.

Don't sneak in technical projects. This is a common and understandable instinct. The team knows something needs to be fixed, they don't think they can get approval for it, so they bury it inside a feature sprint or do it quietly on the side. Resist this. Treat technical projects like features. Put them through the same planning process. Make them visible to product and finance. If you can't articulate why the work is needed and what it costs the business if it doesn't get done, that's a sign you need to sharpen the case, not hide the work. And if you *can* articulate it but leadership still says no, at least the decision is explicit and the risk is documented. Sneaking in work means nobody knows it happened, nobody learns to value it, and the next time something needs attention you're back to sneaking again.

Celebrate the boring wins. Nobody throws a party when you upgrade a database driver or automate a manual deployment step. But these are the wins that prevent the 2 AM phone calls. Acknowledging this work signals to the team that the organization values prevention, not just firefighting.

Warning Signs That Brush Is Becoming Something Worse

A brush fire state can persist for a long time. But certain signals suggest the brush is drying out and conditions are shifting. Watch for these:

Releases are getting slower, not faster. If your deployment cadence is decreasing over time, or if each release requires more coordination than the last, the friction is compounding.

The same systems keep coming up. If every retrospective mentions the same problem, and it still hasn't been addressed, the team has stopped believing it will be. That's how learned helplessness starts.

New hires are taking longer to become productive. This is often the first measurable signal of accumulated organizational knowledge gaps. When onboarding takes longer than it should,

it's because the codebase requires context that's not written down anywhere.

Your best engineers are spending time on maintenance instead of building. If your most experienced people are increasingly occupied with keeping things running rather than creating new value, the balance has shifted. You're consuming capacity on maintenance that should be going toward growth.

You're avoiding areas of the codebase. If there are systems that the team routes around because they're too fragile or too poorly understood to modify safely, that's not caution. That's a section of forest where the brush has gotten too thick to walk through.

The Human Cost You're Not Measuring

In a brush fire state, the team is functional. Projects ship. People collaborate. From the outside, everything looks healthy. And it mostly is. But there's a quiet cost that doesn't show up in any dashboard.

Your team is carrying small workarounds they've stopped noticing. The manual step in the deployment process that takes five minutes. The test that flakes and has to be rerun. The system that nobody touches because it's easier to route around it than to fix it. Each one is minor. None of them register as problems worth raising. But they add up to a persistent, low-grade friction that shapes how the team feels about the work without anyone being able to point to a specific cause.

The bigger risk is what happens to the team's sense of agency. When things are mostly working, the boring problems feel impossible to justify. "We should automate that deployment step" gets met with "it only takes five minutes, we have bigger priorities." "We should refactor that module" gets met with "it works, why touch it?" Over time, people internalize that raising these concerns is not worth the effort. They stop advocating for improvements, not because they've stopped caring, but because the environment has taught them that small problems don't get attention.

This is how a healthy team quietly loses the habit of continuous improvement. Not through crisis, but through comfort. The people who would normally push for better stop pushing because everything is "fine enough." And when conditions eventually change and the organization needs that improvement muscle, it's atrophied.

The teams that stay healthy long-term are the ones where raising a small concern is treated as a contribution, not a complaint. Where fixing a boring problem is recognized as valuable work. Where the team collectively believes they have both the permission and the ability to make things better incrementally. That belief is fragile. It doesn't survive being ignored.

The Choice You Have Right Now

You're in a position most organizations would envy. Your systems work. Your team is functional. You have time.

The question is what you do with that time. You can treat the current stability as evidence that nothing needs to change. Many organizations do. They ride the calm until conditions shift, a key person leaves, a big customer arrives, the market moves, and then they discover that the brush they ignored has become fuel.

Or you can use the stability as an opportunity. Build the controlled burn practice. Clear the debt that's been accumulating. Invest in the boring work that makes your systems more resilient and your team more confident. Do it now, while you can choose the pace and the priority, instead of later when the fire chooses for you.

The best time to address tech debt is when you're not in crisis. You're not in crisis. That's a gift. Use it.

About Understory Collaborative

Understory Collaborative is a team of seasoned technologists who specialize in the work that happens beneath the surface. We're smokejumpers. We help organizations clear the brush before a smolder turns into a bigger fire, and we bring the experience and clarity to help you see what's actually happening on the ground.

Whether you need a focused assessment, hands-on technical leadership, or a partner to help you build the practices that prevent the next crisis, we meet you where you are.

If anything in this paper sounded familiar, we should talk. Reach out at contact@understorycollab.com or visit understorycollab.com/contact.

This white paper is part of the "What's On Fire?" series from Understory Collaborative. Take the self-assessment at [understorycollab.com/quiz] to find out where your organization stands.